Lecture 22: Software Engineering - Game Coding with Pygame

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 30일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

Development Environment Setup

VS Code + Virtual Environment 설정하기

왜 Virtual Environment를 사용할까?

하나의 컴퓨터에서 여러 프로젝트를 진행할 때, 각 프로젝트마다 필요한 라이브러리 버전이 다를 수 있습니다.

문제 상황

- 프로젝트 A: pygame 2.1 필요
- 프로젝트 B: pygame 2.5 필요
- 같은 환경에서 설치하면 충돌 발생

해결 방법: Virtual Environment

프로젝트마다 독립된 Python 환경을 만들어 충돌을 방지합니다.

핵심

하나의 프로젝트 = 하나의 독립된 Python 환경

Step 1: 프로젝트 폴더 생성 및 VS Code 실행

1. 폴더 생성

```
game_project/
```

2. VS Code에서 열기

- VS Code 실행
- File → Open Folder
- game_project 선택

팁

프로젝트마다 폴더를 따로 만드는 습관이 중요합니다.

Step 2: Virtual Environment 생성

명령어 실행 (터미널)

```
python -m venv venv
```

생성 결과

```
game_project/
|-- venv/
```

설명

venv 폴더 안에 Python 실행 환경이 독립적으로 생성됩니다.

Step 3: Virtual Environment 활성화

Windows

```
venv\Scripts\activate
```

Mac / Linux

```
source venv/bin/activate
```

확인 방법

터미널 앞에 (venv) 표시가 나오면 성공입니다.

Step 4: VS Code Python 인터프리터 설정

설정 방법

- Ctrl + Shift + P
- "Python: Select Interpreter"
- venv 선택

중요

이 설정을 하지 않으면 global Python 환경을 사용하게 됩니다.

Step 5: 패키지 설치 (예: pygame)

설치 명령어

```
pip install pygame
```

설치 확인

```
pip list
```

주의

반드시 (venv)가 활성화된 상태에서 설치해야 합니다.

Step 6: main.py 파일 생성

중요 개념

venv는 자동 생성되지만, 코드 파일은 직접 만들어야 합니다.

파일 생성 방법

- VS Code에서 New File 클릭
- 파일 이름: `main.py`

프로젝트 구조

```
game_project/
|-- venv/
|-- main.py
```

핵심

Python은 실행할 파일을 직접 만들어야 합니다.

Step 7: main.py 실행하기

main.py 예제

```
print("Hello, Game World!")
```

실행 방법

```
python main.py
```

확인

정상적으로 출력되면 환경 설정 완료!

정리: 전체 흐름

전체 과정

1. 프로젝트 폴더 생성
2. venv 생성
3. venv 활성화
4. VS Code 인터프리터 설정
5. 패키지 설치
6. main.py 생성
7. 실행 테스트

핵심 메시지

환경 설정이 50%입니다. 코드보다 더 중요합니다.

Pygame Tutorial

Step 1: Quick Start (창 띄우기)

Step 1-1: 기본 코드 작성

목표

화면에 게임 창을 띄우는 것이 목표입니다.

main.py

```
import pygame

pygame.init()

screen = pygame.display.set_mode((1280, 720))
```

설명

이 코드만으로도 게임 창이 생성됩니다.

코드 설명

1. import pygame

pygame 라이브러리를 불러옵니다.

2. pygame.init()

pygame 시스템을 초기화합니다.

3. display.set_mode()

게임 창(window)을 생성합니다.

Step 1-2: 실행해보기

실행 방법

```
python main.py
```

결과

검은 화면 창이 뜨면 성공!

주의

창이 바로 꺼질 수 있습니다 (정상입니다).

왜 창이 바로 꺼질까?

문제

프로그램이 실행되고 바로 종료됩니다.

이유

Python 프로그램은 코드가 끝나면 종료됩니다.

해결 방향

프로그램을 계속 실행시키려면 **반복문(loop)**이 필요합니다.

Step 2: Game Loop

프로그램을 계속 실행시키기

Game Loop 기본 구조

main.py

```
import pygame

pygame.init()
screen = pygame.display.set_mode((1280, 720))

running = True

while running:
  pass
```

핵심

while 루프가 프로그램을 계속 실행시킵니다.

Event 처리 (중요)

문제

닫기 버튼(X)을 눌러도 프로그램이 종료되지 않습니다.

이유

운영체제 이벤트를 처리하지 않았기 때문입니다.

Event 처리 코드 추가

main.py

```
while running:
  for event in pygame.event.get():
    if event.type == pygame.QUIT:
      running = False
```

설명

X 버튼을 누르면 프로그램이 종료됩니다.

Event란 무엇인가?

Event (이벤트)

사용자 또는 시스템에서 발생하는 모든 입력

예시

- 키보드 입력 (W, A, S, D)
- 마우스 클릭
- 창 닫기 버튼 (X)

핵심

프로그램은 입력을 "자동으로" 아는 것이 아니라
이벤트를 통해 전달받습니다.

pygame.event.get()의 의미

개념

운영체제가 발생시킨 이벤트 목록을 가져옵니다.

코드

```
for event in pygame.event.get():
```

설명

이벤트들을 하나씩 꺼내서 확인합니다.

비유

편지함에서 편지를 하나씩 꺼내 읽는 것과 같습니다.

Event 처리 흐름

전체 코드

```
for event in pygame.event.get():
  if event.type == pygame.QUIT:
    running = False
```

동작 과정

1. 이벤트 목록을 가져온다
2. 하나씩 확인한다
3. 종료 이벤트가 있으면 프로그램 종료

핵심

이 구조가 없으면 프로그램은 입력을 처리할 수 없습니다.

화면 업데이트 추가

main.py

```
while running:
  for event in pygame.event.get():
    if event.type == pygame.QUIT:
      running = False

  pygame.display.flip()
```

설명

화면을 계속 업데이트합니다.

왜 화면 업데이트가 필요한가?

문제

화면에 아무것도 보이지 않습니다.

이유

우리는 화면에 직접 그리는 것이 아니라
보이지 않는 공간(버퍼)에 먼저 그립니다.

핵심

그림을 그려도 화면에 바로 보이지 않습니다.

pygame.display.flip()의 역할

개념

보이지 않는 화면을 실제 화면에 출력합니다.

코드

```
pygame.display.flip()
```

비유

종이에 그림을 그리고 나서
그 종이를 사람에게 보여주는 것과 같습니다.

flip()을 하지 않으면?

문제

화면이 갱신되지 않습니다.

결과

- 화면이 멈춘 것처럼 보임
- 변화가 보이지 않음

핵심

flip()은 화면 출력의 마지막 단계입니다.

왜 계속 업데이트할까?

게임의 본질

게임은 계속 변화하는 화면입니다.

예시

- 캐릭터 이동
- 적 움직임
- 애니메이션

핵심

매 프레임마다 화면을 새로 그려야 합니다.

Game Loop 전체 흐름

구조

```
while running:
  # 입력 처리
  # 상태 업데이트
  # 화면 그리기
  pygame.display.flip()
```

핵심

게임은 이 과정을 계속 반복합니다.

pygame.quit()이란?

개념

pygame에서 사용한 시스템을 종료합니다.

코드

```
pygame.quit()
```

설명

window, 입력 시스템 등을 정리합니다.

왜 없어도 프로그램이 종료될까?

현재 코드

`running = False` → while 종료

결과

Python 프로그램 자체가 종료됩니다.

핵심

그래서 pygame.quit()이 없어도 종료는 됩니다.

그럼 왜 pygame.quit()을 쓸까?

이유

- 리소스를 깔끔하게 정리
- 버그 예방
- 프로그램 안정성 확보

핵심

좋은 프로그램은 "정상 종료"를 해야 합니다.

Step 3: Draw Something

화면에 그려보기

Step 3: 목표

목표

화면에 원(circle)을 하나 그립니다.

main.py

```
pygame.draw.circle(screen, "red", (640, 360), 40)
```

설명

화면(screen)에 빨간 원을 그립니다.

코드 설명

draw.circle

- screen: 그릴 위치
- "red": 색상
- (640, 360): 위치 (x, y)
- 40: 반지름

핵심

좌표를 이용해 위치를 지정합니다.

전체 코드 (그리기 포함)

main.py

```
import pygame

pygame.init()
screen = pygame.display.set_mode((1280, 720))

running = True

while running:

  for event in pygame.event.get():
    if event.type == pygame.QUIT:
      running = False

  screen.fill("black")

  pygame.draw.circle(screen, "red", (640, 360), 40)

  pygame.display.flip()

pygame.quit()
```

왜 screen.fill이 필요할까?

문제

이전 프레임의 그림이 남습니다.

해결

`screen.fill("black")`로 화면을 지웁니다.

핵심

매 프레임마다 새로 그려야 합니다.

Game Loop 다시 보기

구조

```
while running:
  # 입력 처리
  # 화면 지우기
  # 그리기
  # 화면 출력
```

핵심

이 구조가 게임의 기본입니다.

Step 4: Move the Object

키보드로 움직이기 (WASD)

Step 4 Overview

main.py (핵심 부분)

```
while running:

  for event in pygame.event.get():
    if event.type == pygame.QUIT:
      running = False

  keys = pygame.key.get_pressed()

  if keys[pygame.K_w]:
    y -= 5

  if keys[pygame.K_s]:
    y += 5

  if keys[pygame.K_a]:
    x -= 5

  if keys[pygame.K_d]:
    x += 5

  screen.fill("black")
```

Step 4: 목표

목표

W, A, S, D 키를 이용해 원을 움직입니다.

main.py

```
x = 640
y = 360
```

설명

원의 위치를 변수로 관리합니다.

x, y 변수는 어디에 선언할까?

위치

```
screen = pygame.display.set_mode((1280, 720))


x = 640
y = 360
running = True
while running:
```

핵심

위치는 loop 밖에서 선언해야 값이 유지됩니다.

draw 코드 수정

main.py

```
pygame.draw.circle(screen, "red", (x, y), 40)
```

핵심

좌표를 변수로 바꾸면 움직일 수 있습니다.

키 입력 받기

main.py

```
keys = pygame.key.get_pressed()
```

설명

현재 눌린 키 상태를 가져옵니다.

WASD 이동 구현

main.py

```
if keys[pygame.K_w]:
  y -= 5

if keys[pygame.K_s]:
  y += 5

if keys[pygame.K_a]:
  x -= 5

if keys[pygame.K_d]:
  x += 5
```

설명

키를 누르면 좌표가 바뀝니다.

핵심 개념 정리

핵심

- 위치는 변수로 관리한다
- 입력에 따라 값을 바꾼다
- 매 프레임 다시 그린다

결론

움직임 = 좌표 변화

Step 5: Player Class

Player 개념으로 확장하기

player.py 생성

목표

Player 클래스를 별도 파일로 분리합니다.

파일 구조

```
game_project/
|-- main.py
|-- player.py
```

핵심

기능별로 파일을 나눕니다.

player.py 코드

player.py

```
import pygame

class Player:
  def __init__(self):
    self.x = 640
    self.y = 360

  def update(self, keys):
    if keys[pygame.K_w]: self.y -= 5
    if keys[pygame.K_s]: self.y += 5
    if keys[pygame.K_a]: self.x -= 5
    if keys[pygame.K_d]: self.x += 5

  def draw(self, screen):
    pygame.draw.circle(screen, "red", (self.x, self.y), 40)
```

main.py 전체 코드 (완성)

main.py

```
import pygame
from player import Player

pygame.init()

screen = pygame.display.set_mode((1280, 720))

player = Player()

running = True

while running:

  for event in pygame.event.get():
    if event.type == pygame.QUIT:
      running = False

  keys = pygame.key.get_pressed()
  player.update(keys)

  screen.fill("black")
  player.draw(screen)

  pygame.display.flip()

pygame.quit()
```

역할 분리

main.py

- 게임 실행
- loop 관리

player.py

- 캐릭터 로직
- 이동 / 그리기

핵심

각 파일은 하나의 역할만 담당합니다.

Step 6: Class 확장하기

Enemy Class

enemy.py 생성

파일 구조

```
game_project/
|-- main.py
|-- player.py
|-- enemy.py
```

핵심

기능별로 파일을 나눕니다.

enemy.py 코드

enemy.py

```
import pygame


class Enemy:
  def __init__(self):
    self.x = 200
    self.y = 200

  def update(self):
    self.x += 1

  def draw(self, screen):
    pygame.draw.circle(screen, "blue", (self.x, self.y), 40)
```

main.py 변경

main.py

```
from player import Player
from enemy import Enemy

player = Player()
enemy = Enemy()

while running:

  keys = pygame.key.get_pressed()
  player.update(keys)
  enemy.update()

  screen.fill("black")
  player.draw(screen)
  enemy.draw(screen)
```

Step 7: 리스트

Multiple Enemies

문제 상황

현재 구조

```
enemy = Enemy()
```

문제

적이 여러 명이라면?

예시

enemy1, enemy2, enemy3 ...

해결 방법: 리스트

main.py

```
enemies = [Enemy(), Enemy(), Enemy()]
```

핵심

여러 객체를 하나로 관리합니다.

update 반복 처리

main.py

```
for enemy in enemies:
  enemy.update()
```

설명

모든 enemy를 반복하면서 업데이트합니다.

draw 반복 처리

main.py

```
for enemy in enemies:
  enemy.draw(screen)
```

핵심

모든 객체를 반복해서 그립니다.

전체 구조 (리스트 포함)

main.py

```
player = Player()

enemies = [Enemy(), Enemy(), Enemy()]

while running:

  keys = pygame.key.get_pressed()
  player.update(keys)

  for enemy in enemies:
    enemy.update()

  screen.fill("black")
  player.draw(screen)

  for enemy in enemies:
    enemy.draw(screen)
```

핵심 개념

핵심

- 객체를 리스트로 관리한다
- 반복문으로 처리한다

결론

게임 = 객체들의 집합

Step 8: Random Position

겹친 문제 해결

문제: Enemy가 겹쳐서 생성됨

현재 코드

```
self.x = 200
self.y = 200
```

문제

모든 Enemy가 같은 위치에서 시작합니다.

해결: 랜덤 위치

enemy.py

```
import random


class Enemy:
  def __init__(self):
    self.x = random.randint(0, 1280)
    self.y = random.randint(0, 720)
```

핵심

Enemy마다 다른 위치에서 시작합니다.

결과

변화

- Enemy들이 화면에 퍼져서 생성됨
- 겹침 문제 해결

핵심

초기값을 다양하게 주면 자연스러운 게임이 됩니다.

충돌이란 무엇인가?

충돌 (Collision)

두 객체가 서로 겹치는 상황

예시

- Player vs Enemy

목표

충돌하면 게임을 종료합니다.

Step 9: Collision & Game Over

충돌 감지와 게임 종료

Rect (사각형)

개념

```
rect = pygame.Rect(x, y, width, height)
```

설명

객체의 위치를 사각형으로 표현합니다.

핵심

Rect는 충돌 감지에 사용됩니다.

Player에 Rect 추가

player.py

```python
def get_rect(self):
    return pygame.Rect(self.x-40, self.y-40, 80, 80)
```

Rect 함수란?

`pygame.Rect(x, y, width, height)`
→ 사각형을 생성하는 함수

코드 해석

- `self.x-40, self.y-40` → 사각형의 왼쪽 위 좌표
- `80, 80` → 사각형의 크기 (가로, 세로)

핵심

원의 중심(self.x, self.y)을 기준으로
사각형을 만들어 충돌을 계산합니다.

Enemy에 Rect 추가

enemy.py

```
def get_rect(self):
  return pygame.Rect(self.x-20, self.y-20, 40, 40)
```

핵심

모든 객체는 Rect를 가져야 합니다.

충돌 감지 코드 (추가)

main.py

```
# # # 이전 코드 # # #
for event in pygame.event.get():
  if event.type == pygame.QUIT:
    running = False


# # # 추가되는 코드 # # #


for enemy in enemies:
  if player.get_rect().colliderect(enemy.get_rect()):
    running = False
```

핵심

Rect가 겹치면 충돌입니다.

전체 흐름

구조

```
while running:
  # 입력
  # 업데이트
  # 충돌 검사
  # 그리기
```

핵심

게임 = 반복 + 조건 + 이벤트

Virtual Environment 종료

명령어

```
deactivate
```

설명

현재 활성화된 virtual environment를 종료합니다.

주의

터미널을 닫아도 자동으로 종료됩니다.

추천 습관

작업이 끝나면 deactivate 하는 것이 좋습니다.

Any Questions?